Architecture Complète du Système NLP — T-AIA-911

Projet : Extraction d'intentions de voyage et génération d'itinéraires ferroviaires
Équipe : T-AIA-911-LIL_3
Date : Février 2026

---

1. Vue d'Ensemble de l'Application

Le système est une application complète permettant à un utilisateur d'entrer une phrase en langage naturel (ex: *"Je voudrais aller de Paris à Lyon"*) et d'obtenir un itinéraire ferroviaire.

Architecture en Couches

```
┌──────────────────────────────────────────────────────────────┐
│                     COUCHE PRÉSENTATION                      │
│   Frontend React/TypeScript (cly/)  — port 5173              │
│   • Interface chat + upload CSV                              │
└──────────────────────────────────────────────────────────────┘
                          │
                          │ HTTP REST
┌──────────────────────────▼───────────────────────────────────┐
│                      COUCHE API                              │
│   FastAPI (api/main.py)  — port 8000                         │
│   • POST /predict   → analyse phrase unique                  │
│   • POST /process_csv → traitement batch                     │
│   • GET  /health    → état du système                        │
└──────────────────────────────────────────────────────────────┘
                          │
                          │
┌──────────────────────────▼───────────────────────────────────┐
│                    COUCHE PIPELINE NLP                       │
│   TravelIntentPipeline (api/pipeline.py)                     │
│                                                              │
│   ┌──────────────┐    ┌──────────────┐    ┌──────────────┐  │
│   │  CLASSIFIER  │──►│     NER      │──►│   PATHFINDING   │  │
│   │  CamemBERT   │    │  CamemBERT   │    │  Dijkstra / A*  │  │
│   │  (valid/inv) │    │ (DEPARTURE / │    │  ou SNCF API    │  │
│   │              │    │   ARRIVAL)   │    │                 │  │
│   └──────────────┘    └──────────────┘    └──────────────┘  │
└──────────────────────────────────────────────────────────────┘
                          │
┌──────────────────────────▼───────────────────────────────────┐
│                    COUCHE DONNÉES                            │
│   • gares-francaises.json (~3000 gares SNCF avec coordonnées)  │
│   • Modèles fine-tunés sauvegardés (classifier/, nlp/models/)  │
│   • SNCF Open Data API (optionnel)                           │
└──────────────────────────────────────────────────────────────┘
```

---

2. Détail des Modules NLP

2.1 Module Classifier (classifier/)

Rôle : Détecter si la phrase est une demande de trajet valide ou non.

Architecture du modèle :

```
Input Text
    │
    ▼
CamemBERT Tokenizer
    │  (max_length=128, padding, truncation)
    ▼
CamemBERT Base (camembert-base)
    │  12 couches Transformer bidirectionnelles
    │  Hidden size: 768
    │  Attention heads: 12
    ▼
[CLS] token representation (768 dimensions)
    │
    ▼
Linear Classifier Layer (768 → 2)
    │
    ▼
Softmax → P(VALID), P(INVALID)
    │
    ▼
Decision: VALID si P(VALID) > 0.5
```

Classe de sortie :

- `VALID` (label=1) : phrase contient une demande d'itinéraire avec départ ET arrivée
- `INVALID` (label=0) : tout le reste

2.2 Module NER (nlp/)

Rôle : Extraire les entités DEPARTURE et ARRIVAL d'une phrase valide.

Architecture du modèle :

```
Input Text
    │
    ▼
CamemBERT Tokenizer
    │  (max_length=128, return_offsets_mapping=True)
    ▼
CamemBERT Base (camembert-base)
    │  12 couches Transformer bidirectionnelles
    ▼
Token Representations (seq_len × 768)
    │
    ▼
Linear Token Classifier (768 → 5)
    │
    ▼
Per-token labels (IOB format):
    O           : pas une entité
    B-DEPARTURE : début d'une ville de départ
    I-DEPARTURE : suite d'une ville de départ
    B-ARRIVAL   : début d'une ville d'arrivée
    I-ARRIVAL   : suite d'une ville d'arrivée
```

Post-processing :

1. Reconstruction des entités multi-tokens (BIO → texte)

2. Filtrage des entités courtes (< 2 caractères)
3. Validation optionnelle contre la liste `gares-francaises.json`

2.3 Module Pathfinding (pathfinding/)

Rôle : Trouver l'itinéraire entre départ et arrivée.

Deux modes :

Mode	Description	Quand
`graph`	Graphe local Haversine + Dijkstra/A*	Défaut, toujours disponible
`sncf_api`	API SNCF temps réel	Si clé API fournie

Architecture du graphe local :

```
gares-francaises.json
    │
    ▼
StationDataLoader
    │  Charge ~3000 gares (nom, lat, lon, segment_drg)
    ▼
GraphBuilder
    │
    ├─► Étape 1: Hubs (segment A) connectés ≤ 500 km (Haversine)
    │
    └─► Étape 2: Gares B/C connectées aux voisines ≤ 200 km
    │
    ▼
Graphe {gare: {voisine: distance_km}}
    │
    ▼
RouteFinder (Dijkstra ou A*)
    │
    ▼
Chemin optimal [Départ, étape1, ..., Arrivée]
```

---

3. Flux de Données — Traitement d'une Phrase

```
Utilisateur: "Je veux prendre le train de Paris pour aller à Lyon"
    │
    ▼
[API] POST /predict → TravelIntentPipeline.predict()
    │
    ▼ Étape 1: CLASSIFIER
    │  Token IDs: [5, 12, 834, 2891, ...]
    │  CamemBERT → [CLS] embedding
    │  Linear → logits [2.1, -1.8]
    │  Softmax → P(VALID)=0.89, P(INVALID)=0.11
    │  Décision: VALID ✓
    │
    ▼ Étape 2: NER
    │  Tokens: ["Je", "veux", "prendre", "le", "train", "de", "Paris", ...]
    │  Labels: [O,   O,      O,         O,   O,       O,   B-DEP,  ...]
    │  "pour", "aller", "à", "Lyon"
```

```
    │   O,      O,      O,   B-ARR
    │  → DEPARTURE = "Paris"
    │  → ARRIVAL   = "Lyon"
    │
    ▼ Étape 3: PATHFINDING (mode graph, algo dijkstra)
    │  "Paris" → Paris (hub A, lat=48.84, lon=2.37)
    │  "Lyon"  → Lyon (hub A, lat=45.75, lon=4.86)
    │  Dijkstra sur graphe → [Paris, Dijon, Lyon]
    │  Distance totale: 463 km
    │  Temps estimé: ~1.65h (TGV @280km/h)
    │
    ▼ Réponse JSON:
    {
      "valid": true,
      "departure": "Paris",
      "arrival": "Lyon",
      "route": ["Paris", "Dijon", "Lyon"],
      "route_distance": 463.0,
      "route_time": 1.65,
      "route_source": "graph",
      "route_algorithm": "dijkstra"
    }
```

---

4. Structure des Fichiers

```
T-AIA-911-LIL_3/
├── api/
│   ├── main.py          # FastAPI endpoints
│   └── pipeline.py      # TravelIntentPipeline (orchestration)
├── classifier/
│   ├── models/          # Modèles fine-tunés CamemBERT classifier
│   └── ...              # Scripts d'entraînement
├── nlp/
│   ├── models/          # Modèles fine-tunés CamemBERT NER
│   └── ...              # Scripts d'entraînement
├── pathfinding/
│   ├── graph_builder.py # Construction du graphe
│   ├── dijkstra.py      # Algorithme Dijkstra
│   ├── astar.py         # Algorithme A*
│   ├── route_finder.py  # Interface principale
│   ├── data_loader.py   # Chargement gares-francaises.json
│   └── utils.py         # Haversine, normalisation
├── dataset/
│   ├── generators/      # Générateurs de données synthétiques
│   └── shared/
│       └── gares-francaises.json
├── cly/                 # Frontend React/TypeScript
└── docs/                # Documentation
```

---

5. Technologies Utilisées

Composant	Technologie	Version
Modèle de base	CamemBERT (camembert-base)	Hugging Face

Framework ML	PyTorch	—
Fine-tuning	HuggingFace Transformers Trainer	—
API Backend	FastAPI + uvicorn	—
Frontend	React + TypeScript + Vite	—
Accélération	MPS (Apple Silicon)	Mac M4
Tokenizer	CamembertTokenizer	—

Document généré le 27/02/2026